

Experiment No. 2: Comparative Study of Low-Code Development Versus Traditional Application Development

1. Aim

To study and compare the Traditional Application Development approach with the Low-Code Development approach based on their development process, speed, cost, flexibility and ease of use.

2. Introduction to Traditional Application Development

Traditional application development is the conventional method of building software by writing source code using programming languages such as Java, Python, C# or JavaScript. Developers manually design, code, test, deploy and maintain the application.

This approach provides complete control over the application's functionality, performance and customization. However, it requires skilled developers, longer development time and higher development costs.

Features

- Manual coding is required.
- High flexibility and customization.
- Suitable for complex applications.
- Requires experienced developers.
- Longer testing and deployment cycles.

3. Introduction to Low-Code Development

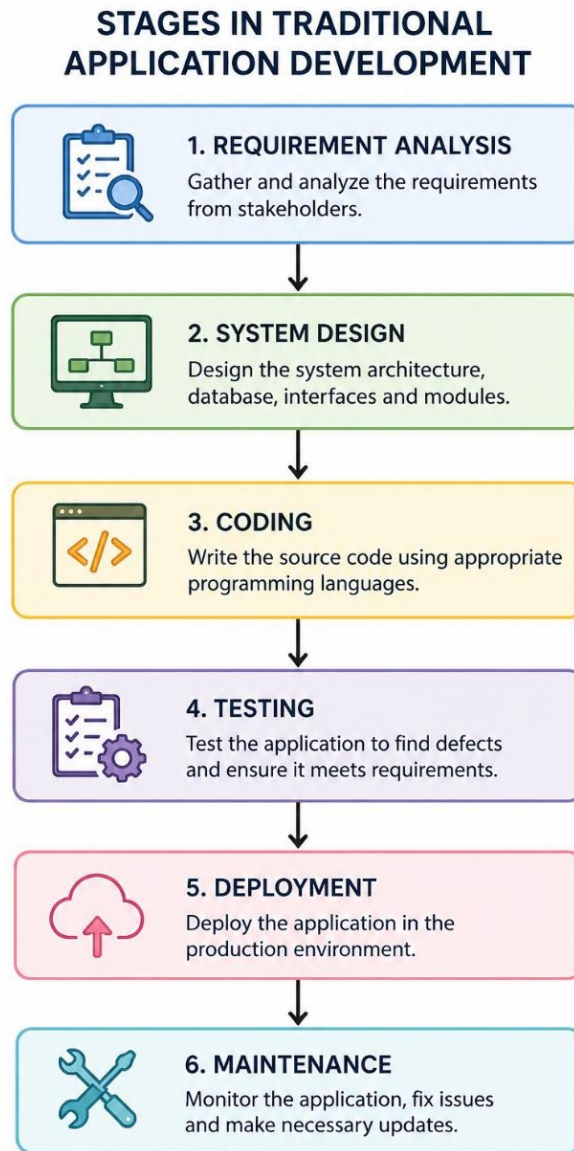
Low-code development is a modern software development approach that enables applications to be built using graphical user interfaces, drag-and-drop components and minimal hand-written code.

It allows professional developers and non-technical users to create applications quickly, reducing development time and effort.

Features

- Drag-and-drop interface.
- Minimal coding required.
- Faster application development.
- Easy integration with existing systems.
- Suitable for rapid application development and business applications.

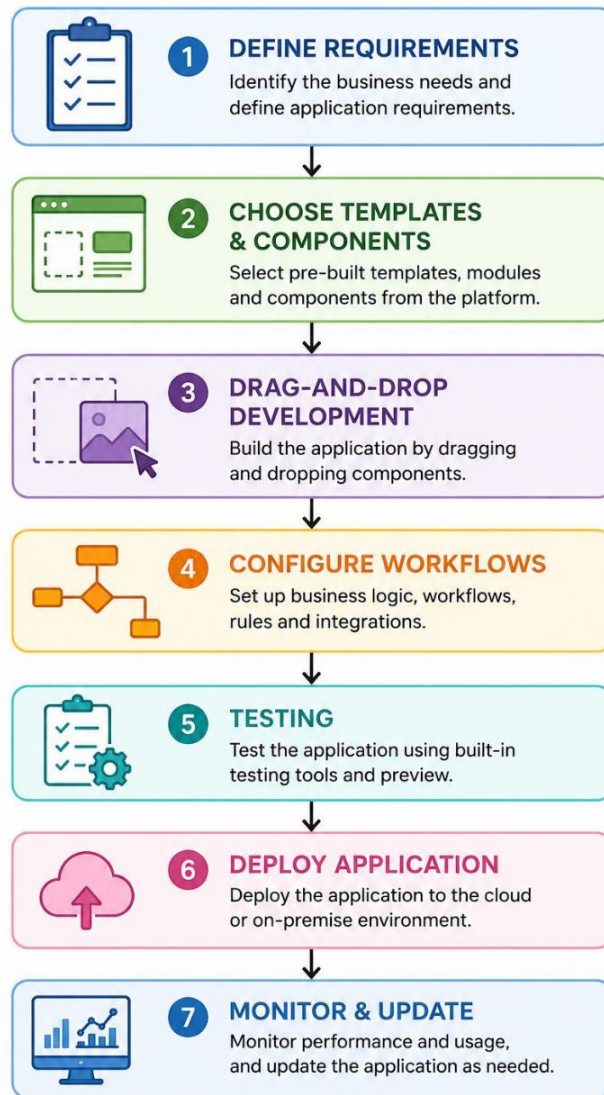
4. Diagram for Stages in Traditional Application Development



The stages shown are requirement analysis, system design, coding, testing, deployment and maintenance.

5. Diagram for Steps in Low-Code Development

STEPS IN LOW-CODE DEVELOPMENT



The low-code process includes defining requirements, choosing templates and components, drag-and-drop development, configuring workflows, testing, deployment and monitoring/updating.

6. Comparative Analysis of Traditional and Low-Code Development

Parameter	Traditional Development	Low-Code Development
Coding	Extensive manual coding	Minimal coding
Development Speed	Slow	Very Fast
Development Cost	High	Low
Required Skills	Professional programmers	Developers and non-developers
Customization	Very High	Moderate to High
Learning Curve	Steep	Easy
Maintenance	Time-consuming	Easier
Testing	Mostly manual	Automated tools available
Deployment	Takes more time	Quick deployment
Best Suitable For	Large and complex applications	Business applications and rapid development

7. Main Differences

Development: Traditional development depends mainly on manual programming, whereas low-code development uses visual components and less handwritten code.

Time and Cost: Traditional development generally requires more development and testing effort. Low-code platforms are designed to reduce development time and effort.

Skills: Traditional development normally requires programming knowledge. Low-code platforms can also support users with less programming experience.

Customization: Traditional development gives extensive control over application behaviour. Low-code platforms provide configurable components and can also support additional logic where required.

8. Advantages and Limitations

Traditional Development	Low-Code Development
High control over application architecture	Faster application development
Suitable for highly customized systems	Drag-and-drop development
Suitable for complex applications	Less manual coding
Requires skilled developers	Can be used by developers and non-developers
Longer development and maintenance effort	Simplifies many development tasks

9. Result

The comparative study was completed. Traditional Application Development provides extensive control and customization but generally involves more manual coding, development effort and skilled programming. Low-Code Development uses visual components and minimal coding to simplify and speed up application development.

The suitable approach depends on the application requirements, complexity, available skills, budget and development time.